

Sprint 9 Notes: Tables and Debugging Valid HTML

Sprint 9 Goal

The goal of Sprint 9 is to learn how to create HTML tables correctly and how to find/fix structural HTML mistakes.

In simple words:

Sprint 9 teaches you two big skills:

1. How to show data in rows and columns using HTML tables.
2. How to debug broken HTML using validators and browser DevTools.

By the end of this sprint, you should be able to:

- Use tables only for real tabular data.
- Build a table using `table`, `caption`, `thead`, `tbody`, `tfoot`, `tr`, `th`, and `td`.
- Use `th` for header cells and `td` for normal data cells.
- Use `colspan` and `rowspan` when needed.
- Avoid using tables for page layout.
- Understand why browsers may hide HTML mistakes.
- Use an HTML validator to check your code.
- Use DevTools Elements and Console to inspect problems.
- Fix broken table structure and missing closing tags.

1. Where Sprint 9 Fits in Your HTML Progress

Before Sprint 9, you already learned many important HTML foundations.

Sprint 1

You learned basic HTML syntax:

```
<h1>My Page</h1>
<p>This is a paragraph.</p>

```

You learned:

- elements

- tags
- attributes
- void elements
- `id`
- `class`
- basic image structure

Sprint 2

You learned the full HTML document structure:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Page</title>
</head>
<body>
  <h1>My Page</h1>
</body>
</html>
```

You learned:

- DOCTYPE
- `html`
- `head`
- `body`
- metadata
- visible page content

Sprint 3

You learned text meaning:

```
<p><strong>Warning:</strong> Do not share your password.</p>
<p>The term <i>semantic HTML</i> means HTML with meaning.</p>
```

You learned to choose HTML elements based on meaning, not appearance.

Sprint 4

You learned links and navigation:

```
<nav>
  <a href="index.html">Home</a>
  <a href="about.html">About</a>
  <a href="contact.html">Contact</a>
</nav>
```

You learned:

- a
- href
- relative paths
- absolute URLs
- tel:
- mailto:
- address
- nav

Sprint 5

You learned media:

```

```

You learned:

- images
- audio
- video
- SVG
- iframe
- media responsibility

Sprint 6

You learned semantic page structure:

```
<header>
  <h1>Code Learning Hub</h1>
</header>

<main>
  <section>
    <h2>Featured Courses</h2>
    <p>Learn HTML step by step.</p>
```

```
</section>
</main>
```

You learned:

- header
- nav
- main
- section
- article
- aside
- footer
- heading hierarchy

Sprint 7

You learned form fundamentals:

```
<form action="">
  <label for="email">Email Address:</label>
  <input type="email" id="email" name="email">
</form>
```

You learned:

- form
- action
- input
- label
- for
- id
- name
- placeholder

Sprint 8

You learned buttons, validation, and form states:

```
<input type="email" id="email" name="email" required>
<button type="submit">Submit</button>
```

You learned:

- button

- type="submit"
- type="reset"
- type="button"
- required
- minlength
- maxlength
- disabled
- readonly

Sprint 9

Now you are learning tables and debugging.

Simple idea:

Sprints 1 to 8 taught you how to build pages and forms. Sprint 9 teaches you how to show data correctly and check whether your HTML structure is valid.

2. What Is a Table?

A table is used to show data in rows and columns.

Think of a table like a small spreadsheet.

Example data:

Course	Duration	Level
HTML	10 hours	Beginner
CSS	15 hours	Beginner
JavaScript	25 hours	Intermediate

This kind of content is good for a table because it has clear rows and columns.

Simple Rule

Use a table when your content naturally fits into rows and columns.

3. Good Uses of Tables

Tables are good for structured data.

Examples:

Content Type	Good for a Table?	Why?
Course comparison	Yes	It compares course name, duration, and level.
Student marks	Yes	It has students, subjects, and marks.
Product prices	Yes	It has product names, prices, and quantities.
Timetable	Yes	It has days, times, and activities.
Bank transactions	Yes	It has dates, descriptions, and amounts.
Employee records	Yes	It has names, roles, and departments.

Example:

```
<table>
  <caption>Student Marks</caption>
  <thead>
    <tr>
      <th>Student</th>
      <th>Subject</th>
      <th>Marks</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Kamal</td>
      <td>HTML</td>
      <td>85</td>
    </tr>
  </tbody>
</table>
```

This is a good table because the data has rows and columns.

4. Bad Uses of Tables

Do not use tables to design the page layout.

Bad Example

```
<table>
  <tr>
    <td>Sidebar</td>
    <td>Main content</td>
  </tr>
</table>
```

This is weak HTML because the table is not showing real data. It is only being used to place content side by side.

Better Semantic Structure

```
<main>
  <aside>
    <h2>Related Links</h2>
    <p>Extra information goes here.</p>
  </aside>

  <section>
    <h2>Main Content</h2>
    <p>Main page content goes here.</p>
  </section>
</main>
```

This is better because:

- `main` means main page content.
- `aside` means related side content.
- `section` means a meaningful content group.

Simple Rule

Use tables for data. Do not use tables for layout.

5. Main Table Elements

Sprint 9 focuses on these main table elements:

Element	Simple Meaning
<code>table</code>	The whole table container
<code>caption</code>	The table title or explanation
<code>thead</code>	The table header area
<code>tbody</code>	The main table data area
<code>tfoot</code>	The table footer or summary area
<code>tr</code>	One table row
<code>th</code>	A table header cell
<code>td</code>	A normal table data cell

Now let us understand each one slowly.

6. The `table` Element

The `table` element is the outer container for the whole table.

Basic pattern:

```
<table>
  ...
</table>
```

Everything related to the table goes inside the `table` element.

Example:

```
<table>
  <tr>
    <th>Course</th>
    <th>Duration</th>
  </tr>
  <tr>
    <td>HTML</td>
    <td>10 hours</td>
  </tr>
</table>
```


Simple meaning:

`table` tells the browser that this content is tabular data.

7. The `caption` Element

The `caption` element gives the table a title or short explanation.

Example:

```
<table>
  <caption>Course Comparison</caption>
</table>
```

The caption explains what the table is about.

Why `caption` Is Useful

A caption helps:

- users understand the table quickly
- screen reader users understand the table purpose
- developers understand the table in the code
- the table feels more complete and meaningful

Correct Placement

The `caption` should be placed directly after the opening `table` tag.

Good:

```
<table>
  <caption>Course Comparison</caption>
  ...
</table>
```

Weak:

```
<table>
  <thead>
    <tr>
```

```
<th>Course</th>
</tr>
</thead>
<caption>Course Comparison</caption>
</table>
```

The second version is weak because the caption should come near the start of the table.

Simple Rule

Use `caption` to name or describe the table.

8. The `thead` Element

The `thead` element contains the header rows of a table.

Example:

```
<thead>
  <tr>
    <th>Course</th>
    <th>Duration</th>
    <th>Level</th>
  </tr>
</thead>
```

Simple meaning:

`thead` tells the browser:

This part contains the table headings.

The `thead` usually contains cells such as:

- Course
- Duration
- Level
- Name
- Price
- Quantity
- Date
- Amount

Simple Rule

Use `thead` for the table heading area.

9. The `tbody` Element

The `tbody` element contains the main data rows of the table.

Example:

```
<tbody>
  <tr>
    <td>HTML</td>
    <td>10 hours</td>
    <td>Beginner</td>
  </tr>
</tbody>
```

Simple meaning:

`tbody` tells the browser:

This is the main data area of the table.

Most table rows usually go inside `tbody`.

Simple Rule

Use `tbody` for the main table data.

10. The `tfoot` Element

The `tfoot` element contains footer or summary information for the table.

Example:

```
<tfoot>
  <tr>
    <td colspan="3">Course times are estimated.</td>
```

```
</tr>
</tfoot>
```

The footer might contain:

- totals
- notes
- summary information
- table metadata
- estimated value explanation

Example use cases:

Table Type	Possible <code>tfoot</code> Content
Sales table	Total sales amount
Course table	Course times are estimated
Invoice table	Grand total
Attendance table	Total present days

Simple Rule

Use `tfoot` for summary or footer information related to the table.

11. The `tr` Element

The `tr` element creates one table row.

Example:

```
<tr>
  <td>HTML</td>
  <td>10 hours</td>
  <td>Beginner</td>
</tr>
```

Simple meaning:

`tr` means table row.

A table row contains table cells.

Those cells can be:

- `th` header cells
- `td` data cells

Important Rule

Every `th` and `td` should be inside a `tr`.

Good:

```
<tr>
  <td>HTML</td>
  <td>10 hours</td>
</tr>
```

Bad:

```
<table>
  <td>HTML</td>
  <td>10 hours</td>
</table>
```

The bad version has `td` elements without a row.

Simple Rule

Use `tr` to create each row in the table.

12. The `th` Element

The `th` element creates a table header cell.

Example:

```
<th>Course</th>
```

Use `th` for cells that act as headings.

Examples:

- Course
- Duration
- Level
- Name
- Price
- Total
- Date
- Amount

Full example:

```
<tr>
  <th>Course</th>
  <th>Duration</th>
  <th>Level</th>
</tr>
```

Here, the row contains column headings.

Row Header Example

Sometimes the first cell of each row can also be a header.

Example:

```
<tr>
  <th>HTML</th>
  <td>10 hours</td>
  <td>Beginner</td>
</tr>
```

Here, `HTML` is the row heading. The other cells are data about HTML.

Simple Rule

Use `th` when the cell is a heading or label for other cells.

13. The `td` Element

The `td` element creates a normal table data cell.

Example:

```
<td>10 hours</td>
```

Use `td` for actual data values.

Examples:

- 10 hours
- Beginner
- \$20
- Colombo
- 2026-05-28
- 85 marks

Full example:

```
<tr>
  <td>HTML</td>
  <td>10 hours</td>
  <td>Beginner</td>
</tr>
```

Simple Rule

Use `td` for normal data cells.

14. `th` vs `td`

Beginners often confuse `th` and `td`.

They are not the same.

Element	Meaning	Use For
<code>th</code>	Header cell	Headings or labels
<code>td</code>	Data cell	Normal data

Example

```
<tr>
  <th>Course</th>
  <th>Duration</th>
  <th>Level</th>
</tr>
<tr>
  <td>HTML</td>
  <td>10 hours</td>
  <td>Beginner</td>
</tr>
```

In the first row:

- `Course` is a heading.
- `Duration` is a heading.
- `Level` is a heading.

So they use `th`.

In the second row:

- `HTML` is data.
- `10 hours` is data.
- `Beginner` is data.

So they use `td`.

Simple Memory Rule

`th` = table heading

`td` = table data

15. Complete Beginner Table Structure

This is the main Sprint 9 table pattern.

```
<table>
  <caption>Course Comparison</caption>

  <thead>
```



```

    <tr>
      <th>Course</th>
      <th>Duration</th>
      <th>Level</th>
    </tr>
  </thead>

  <tbody>
    <tr>
      <td>HTML</td>
      <td>10 hours</td>
      <td>Beginner</td>
    </tr>

    <tr>
      <td>CSS</td>
      <td>15 hours</td>
      <td>Beginner</td>
    </tr>

    <tr>
      <td>JavaScript</td>
      <td>25 hours</td>
      <td>Intermediate</td>
    </tr>
  </tbody>

  <tfoot>
    <tr>
      <td colspan="3">Course times are estimated.</td>
    </tr>
  </tfoot>
</table>

```

Explanation

```
<table>
```

Starts the table.

```
<caption>Course Comparison</caption>
```

Gives the table a title.

```
<thead>
```

Starts the header area.

```
<tr>
```

Starts a row.

```
<th>Course</th>
```

Creates a header cell.

```
<tbody>
```

Starts the main data area.

```
<td>HTML</td>
```

Creates a normal data cell.

```
<tfoot>
```

Starts the footer area.

```
<td colspan="3">Course times are estimated.</td>
```

Creates one footer cell that stretches across three columns.

16. Simple Table Without `thead`, `tbody`, and `tfoot`

A very small table can work without `thead`, `tbody`, and `tfoot`.

Example:

```

<table>
  <caption>Simple Course Table</caption>
  <tr>
    <th>Course</th>
    <th>Duration</th>
  </tr>
  <tr>
    <td>HTML</td>
    <td>10 hours</td>
  </tr>
</table>

```

This can work.

But for Sprint 9 practice, you should use the more complete structure:

- `caption`
- `thead`
- `tbody`
- `tfoot`

This helps you build cleaner and more organized tables.

17. The `colspan` Attribute

`colspan` means a cell should stretch across multiple columns.

Example

```

<td colspan="3">Course times are estimated.</td>

```

This means:

This one cell covers 3 columns.

Full Example

```

<table>
  <caption>Course Summary</caption>
  <thead>
    <tr>

```

```

        <th>Course</th>
        <th>Duration</th>
        <th>Level</th>
    </tr>
</thead>
<tbody>
    <tr>
        <td>HTML</td>
        <td>10 hours</td>
        <td>Beginner</td>
    </tr>
</tbody>
<tfoot>
    <tr>
        <td colspan="3">This table is for beginner HTML practice.</td>
    </tr>
</tfoot>
</table>

```

There are 3 columns:

1. Course
2. Duration
3. Level

So this footer cell uses:

```
colspan="3"
```

Another `colspan` Example

```

<table id="quickfacts">
  <thead>
    <tr>
      <th colspan="2">Quick Facts</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th>2023 Median Pay</th>
      <td>$130,160 per year</td>
    </tr>
  </tbody>
</table>

```

Here, `Quick Facts` stretches across 2 columns.

Simple Rule

Use `colspan` when one cell needs to cover more than one column.

18. The `rowspan` Attribute

`rowspan` means a cell should stretch down across multiple rows.

Example

```
<table>
  <caption>Frontend Topics</caption>
  <tbody>
    <tr>
      <th rowspan="2">Frontend</th>
      <td>HTML</td>
    </tr>
    <tr>
      <td>CSS</td>
    </tr>
  </tbody>
</table>
```

This means:

The `Frontend` header cell covers 2 rows.

Visual Idea

Frontend	HTML
Frontend continues	CSS

The word `Frontend` belongs to both HTML and CSS rows.

Simple Rule

Use `rowspan` when one cell needs to cover more than one row.

19. colspan vs rowspan

Attribute	Meaning	Direction
colspan	Cell stretches across columns	Left to right
rowspan	Cell stretches across rows	Top to bottom

Memory Rule

colspan = column span = across columns

rowspan = row span = across rows

Beginner Warning

Do not overuse colspan and rowspan.

Large complicated tables are harder to read, harder to maintain, and harder for assistive technology users.

Keep tables simple whenever possible.

20. Why Tables Should Stay Simple

Tables can become difficult when they have:

- many nested headers
- many merged cells
- too many columns
- too many rows
- complicated colspan and rowspan

For beginners, simple tables are better.

Good beginner table:

```
<table>
  <caption>Course List</caption>
  <thead>
    <tr>
      <th>Course</th>
      <th>Duration</th>
      <th>Level</th>
```

```
</tr>
</thead>
<tbody>
  <tr>
    <td>HTML</td>
    <td>10 hours</td>
    <td>Beginner</td>
  </tr>
</tbody>
</table>
```

Weak beginner table:

```
<table>
  <tr>
    <th rowspan="4" colspan="2">Very Complex Header</th>
    <td>Data</td>
  </tr>
</table>
```

The second version is not wrong only because it uses `rowspan` and `colspan`, but it is difficult for a beginner to understand and debug.

Simple Rule

Use merged cells only when the data really needs them.

21. Table Accessibility Introduction

Sprint 9 gives an introduction to table accessibility. You will study accessible table patterns more deeply in Sprint 11.

For now, remember these simple accessibility ideas.

1. Use tables only for tabular data

Screen readers may announce table rows and columns to users.

If you use a table only for page layout, the user may hear a confusing table structure that does not represent real data.

2. Use `caption`

A caption tells users what the table is about.

```
<caption>Course Comparison</caption>
```

This is better than using a random paragraph or `div` as the table title.

3. Use `th` for headers

Header cells help users understand what each row or column means.

```
<th>Course</th>  
<th>Duration</th>  
<th>Level</th>
```

4. Keep tables simple

Simple tables are easier for everyone to understand.

Beginner Accessibility Rule

A good beginner table has:

- a meaningful `caption`
- clear `th` cells
- simple row and column structure
- no layout-table usage

22. Do Not Fake Tables with `div`

Do not create fake data tables using only `div` elements.

Weak Example

```
<div class="table">  
  <div class="row">  
    <div class="cell">Course</div>  
    <div class="cell">Duration</div>  
  </div>
```



```
<div class="row">
  <div class="cell">HTML</div>
  <div class="cell">10 hours</div>
</div>
</div>
```

This may look like a table after CSS, but HTML does not understand it as a real table.

Better Example

```
<table>
  <caption>Course Comparison</caption>
  <thead>
    <tr>
      <th>Course</th>
      <th>Duration</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>HTML</td>
      <td>10 hours</td>
    </tr>
  </tbody>
</table>
```

This is better because the HTML structure matches the meaning of the content.

Simple Rule

If the content is real tabular data, use a real HTML table.

23. Common Table Mistakes and Fixes

Mistake 1: Using `div` as a Table Title

Weak:

```
<table>
  <div>Course Table</div>
</table>
```

Problem:

A `div` should not be used as the table title inside a table.

Better:

```
<table>
  <caption>Course Table</caption>
</table>
```

Why better?

`caption` clearly means table title.

Mistake 2: Putting `td` Directly Inside `table`

Weak:

```
<table>
  <td>HTML</td>
  <td>10 hours</td>
</table>
```

Problem:

The data cells are not inside a row.

Better:

```
<table>
  <tr>
    <td>HTML</td>
    <td>10 hours</td>
  </tr>
</table>
```

Why better?

Every `td` belongs inside a `tr`.

Mistake 3: Using `td` for Header Cells

Weak:

```
<tr>
  <td>Name</td>
  <td>Hours</td>
</tr>
```

Problem:

`Name` and `Hours` are headings, not normal data.

Better:

```
<tr>
  <th>Name</th>
  <th>Hours</th>
</tr>
```

Why better?

`th` tells the browser these cells are headers.

Mistake 4: Putting Paragraphs Directly Inside a Table

Weak:

```
<table>
  <p>HTML</p>
  <td>10</td>
</table>
```

Problem:

A paragraph should not be placed directly inside a table like this.

Better:

```
<table>
  <tr>
    <td>HTML</td>
    <td>10</td>
  </tr>
</table>
```

Why better?

The content is now inside table cells, and the cells are inside a row.

Mistake 5: Using Tables for Layout

Weak:

```
<table>
  <tr>
    <td>Menu</td>
    <td>Page content</td>
  </tr>
</table>
```

Problem:

This is layout, not data.

Better:

```
<header>
  <nav>
    <a href="index.html">Home</a>
    <a href="about.html">About</a>
  </nav>
</header>

<main>
  <section>
    <h2>Page Content</h2>
    <p>This is the main content.</p>
  </section>
</main>
```

Why better?

Semantic HTML describes the meaning of the page parts.

24. What Is Debugging?

Debugging means finding and fixing problems in your code.

In Sprint 9, debugging means checking problems such as:

- missing closing tags
- wrong nesting
- table cells outside rows
- incorrect table structure
- broken links
- elements in the wrong place
- invalid HTML that browsers try to fix automatically

Simple meaning:

Debugging is the process of asking: What is broken, why is it broken, and how can I fix it?

25. Browsers Are Forgiving

Browsers are forgiving.

This means the browser often tries to show the page even when your HTML is wrong.

That sounds helpful, but it can hide mistakes.

Example

Broken:

```
<h2>Notes  
<p>This is not meant to be heading text.</p>
```

Problem:

The `h2` heading is not closed.

The browser may guess the structure incorrectly.

It may treat the paragraph as part of the heading.

Correct:

```
<h2>Notes</h2>
<p>This is not meant to be heading text.</p>
```

Why This Matters

The page may still appear in the browser, but the structure may be wrong.

Wrong structure can affect:

- accessibility
- screen reader navigation
- search engine understanding
- styling
- JavaScript behavior later
- developer understanding

Simple Rule

Do not trust only what you see on the page. Check the actual HTML structure.

26. Source HTML vs Parsed DOM

This is a very important debugging idea.

Source HTML

Source HTML is the code you wrote in your file.

Example:

```
<h2>Notes
<p>This is a paragraph.</p>
```

Parsed DOM

The parsed DOM is the structure the browser creates after reading your HTML.

If your HTML is broken, the browser may create a different structure from what you expected.

Simple meaning:

- Source HTML = what you wrote
- Parsed DOM = what the browser understood

Why This Matters

Your source code may contain a mistake.

The browser may silently fix or rearrange it.

That is why DevTools is useful.

DevTools shows the structure as the browser understands it.

27. HTML Validator

A validator checks your HTML against official HTML rules.

It helps you find mistakes before they become bigger problems.

A common validator is:

validator.w3.org

What a Validator Can Find

A validator can help find:

- missing closing tags
- invalid nesting
- table cells outside table rows
- wrong element placement
- missing required document structure
- invalid attributes
- broken table structure

Example Validator Problem

Broken:

```
<table>
  <td>HTML</td>
</table>
```

A validator can warn that `td` is not correctly placed.

Better:

```
<table>
  <tr>
    <td>HTML</td>
  </tr>
</table>
```

Simple Rule

Use a validator to check whether your HTML follows standards.

28. Why Validators Are Important

A browser may show broken HTML, but a validator tells you the truth about the structure.

Browser Thinking

The browser may think:

I will try to display this page somehow.

Validator Thinking

The validator says:

This HTML does not follow the rules. Fix it.

Simple Example

Broken:

```
<h1>Article Topic</h1>
<h2>Subheading 1</h2>
<h2>Subheading 2</h2>
<h2>Subheading 3
<p>This paragraph may be parsed as part of the h2.</p>
```

Problem:

The third `h2` is not closed.

Correct:

```
<h1>Article Topic</h1>
<h2>Subheading 1</h2>
<h2>Subheading 2</h2>
<h2>Subheading 3</h2>
<p>This paragraph is now separate from the h2.</p>
```

Simple Rule

If the validator shows an error, read it carefully and fix the structure.

29. Browser DevTools

DevTools means developer tools built into the browser.

You can usually open DevTools by:

- right-clicking the page and choosing **Inspect**
- pressing `F12`
- pressing `Ctrl + Shift + I` on Windows
- pressing `Cmd + Option + I` on Mac

DevTools helps you see what the browser understands.

Sprint 9 focuses mainly on:

- Elements tab

- Console tab
 - DOM inspector
-

30. DevTools Elements Tab

The Elements tab shows the browser-parsed HTML structure.

This is useful because the browser may change or repair broken HTML.

What You Can Check in Elements

You can check:

- whether headings are closed correctly
- whether a paragraph is accidentally inside a heading
- whether table cells are inside rows
- whether table sections are created correctly
- whether the browser moved invalid elements somewhere else

Example

You write:

```
<h2>Notes  
<p>This is not meant to be heading text.</p>
```

In Elements, you may see the browser has interpreted the structure in a different way.

That tells you your source HTML needs to be fixed.

Simple Rule

Use the Elements tab to inspect the HTML structure the browser actually created.

31. DOM Inspector

DOM means Document Object Model.

In simple words:

The DOM is the browser's internal structure of your HTML page.

The DOM inspector lets you inspect the page as the browser understands it.

Simple Mental Model

Your HTML file is like a written recipe.

The DOM is like the actual dish the browser made from the recipe.

If the recipe has mistakes, the browser may still try to make something, but the final structure may not be what you expected.

Simple Rule

Use the DOM inspector to compare what you wrote with what the browser built.

32. DevTools Console Tab

The Console tab shows messages, warnings, and errors.

In Sprint 9, the Console is useful for spotting broken resources or links.

Broken Link Example

```
<a href="https://www.freecodecamp.og">Visit freeCodeCamp</a>
```

Problem:

The URL is typed incorrectly.

It should be:

```
<a href="https://www.freecodecamp.org">Visit freeCodeCamp</a>
```

The Console or browser network tools may show an error if a resource cannot be found.

404 Error

A 404 error means:

The browser tried to find a file or page, but it could not find it.

Common causes:

- wrong file name
- wrong folder path
- spelling mistake in URL
- missing image file
- missing CSS file
- broken link

Simple Rule

Use the Console to notice errors and broken paths.

33. Debugging Workflow for Sprint 9

When your HTML does not work correctly, follow this order.

Step 1: Look at the visible page

Ask:

- Does the table display?
- Are rows and columns correct?
- Is anything missing?
- Is text appearing in the wrong place?

Step 2: Inspect with DevTools Elements

Ask:

- Did the browser understand my structure correctly?
- Are `td` and `th` inside `tr`?
- Is the paragraph accidentally inside a heading?
- Did the browser move invalid table content?

Step 3: Check the Console

Ask:

- Are there 404 errors?
- Are there broken file paths?
- Are resources missing?

Step 4: Run the HTML validator

Ask:

- Are there structural HTML errors?
- Are elements nested incorrectly?
- Are table cells in the wrong place?
- Are any tags missing?

Step 5: Fix one problem at a time

Do not try to fix everything randomly.

Fix one error, save the file, refresh the browser, and check again.

Simple Rule

Debug slowly and carefully. One fixed structure problem can solve many visible problems.

34. Sprint 9 Broken Code Practice

Broken Code

```
<table>
  <div>Course Table</div>
  <tr>
    <td>Name</td>
    <td>Hours</td>
  </tr>
  <p>HTML</p>
  <td>10</td>
</table>

<h2>Notes
<p>This is not meant to be heading text.</p>
```

This code has several problems.

35. Problems in the Broken Code

Problem 1: `div` is inside the table

Broken:

```
<table>
  <div>Course Table</div>
</table>
```

Problem:

A `div` is not the correct element for a table title.

Better:

```
<table>
  <caption>Course Table</caption>
</table>
```

Problem 2: Header cells use `td`

Broken:

```
<tr>
  <td>Name</td>
  <td>Hours</td>
</tr>
```

Problem:

`Name` and `Hours` are headings, so they should use `th`.

Better:

```
<tr>
  <th>Name</th>
```

```
<th>Hours</th>
</tr>
```

Problem 3: Paragraph is directly inside the table

Broken:

```
<table>
  <p>HTML</p>
</table>
```

Problem:

A paragraph should not be placed directly inside a table like this.

Better:

```
<tr>
  <td>HTML</td>
  <td>10</td>
</tr>
```

Problem 4: `td` is outside a `tr`

Broken:

```
<td>10</td>
```

Problem:

A `td` must be inside a `tr`.

Better:

```
<tr>
  <td>HTML</td>
```

```
<td>10</td>
</tr>
```

Problem 5: `h2` is not closed

Broken:

```
<h2>Notes
<p>This is not meant to be heading text.</p>
```

Problem:

The heading is not closed.

Better:

```
<h2>Notes</h2>
<p>This is not meant to be heading text.</p>
```

36. Fixed Version of the Broken Code

```
<table>
  <caption>Course Table</caption>

  <thead>
    <tr>
      <th>Name</th>
      <th>Hours</th>
    </tr>
  </thead>

  <tbody>
    <tr>
      <td>HTML</td>
      <td>10</td>
    </tr>
  </tbody>
</table>
```



```
<h2>Notes</h2>
<p>This is not meant to be heading text.</p>
```

Why This Is Correct

- `caption` is used for the table title.
- `thead` contains the heading row.
- `th` is used for `Name` and `Hours`.
- `tbody` contains the main data row.
- `td` cells are inside a `tr`.
- The `h2` heading is closed correctly.
- The paragraph is separate from the heading.

37. Full Sprint 9 Example Page

Create a file named:

```
sprint-9-course-comparison.html
```

Example code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Course Comparison Table</title>
  <meta name="description" content="A beginner HTML page showing a course
comparison table.">
</head>
<body>
  <header>
    <h1>Course Comparison Table</h1>
  </header>

  <main>
    <p>This page compares beginner web development courses.</p>

    <table>
      <caption>Beginner Web Development Course Comparison</caption>

      <thead>
```

```

        <tr>
            <th>Course</th>
            <th>Duration</th>
            <th>Level</th>
            <th>Main Skill</th>
        </tr>
    </thead>

    <tbody>
        <tr>
            <td>HTML</td>
            <td>10 hours</td>
            <td>Beginner</td>
            <td>Page structure</td>
        </tr>

        <tr>
            <td>CSS</td>
            <td>15 hours</td>
            <td>Beginner</td>
            <td>Page styling</td>
        </tr>

        <tr>
            <td>JavaScript</td>
            <td>25 hours</td>
            <td>Intermediate</td>
            <td>Page behavior</td>
        </tr>
    </tbody>

    <tfoot>
        <tr>
            <td colspan="4">Course times are estimated for beginner practice.</td>
        </tr>
    </tfoot>
</table>
</main>
</body>
</html>

```

38. Explanation of the Full Example

Boilerplate

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Course Comparison Table</title>
</head>
<body>
  ...
</body>
</html>
```

This creates a complete HTML document.

Header

```
<header>
  <h1>Course Comparison Table</h1>
</header>
```

This gives the page a clear main heading.

Main Content

```
<main>
  <p>This page compares beginner web development courses.</p>
  ...
</main>
```

The `main` element contains the main content of the page.

Table Start

```
<table>
```

This starts the table.

Caption

```
<caption>Beginner Web Development Course Comparison</caption>
```

This tells users what the table is about.

Table Head

```
<thead>
  <tr>
    <th>Course</th>
    <th>Duration</th>
    <th>Level</th>
    <th>Main Skill</th>
  </tr>
</thead>
```

This creates the header area and heading row.

Table Body

```
<tbody>
  <tr>
    <td>HTML</td>
    <td>10 hours</td>
    <td>Beginner</td>
    <td>Page structure</td>
  </tr>
</tbody>
```

This creates the main data rows.

Table Footer

```
<tfoot>
  <tr>
    <td colspan="4">Course times are estimated for beginner practice.</td>
  </tr>
</tfoot>
```

This creates a footer note that stretches across 4 columns.

39. Another Example: Quick Facts Table

```
<table id="quickfacts">
  <caption>Software Developer Quick Facts</caption>

  <thead>
    <tr>
      <th colspan="2">Quick Facts</th>
    </tr>
  </thead>

  <tbody>
    <tr>
      <th>Median Pay</th>
      <td>$130,160 per year</td>
    </tr>

    <tr>
      <th>Education</th>
      <td>Computer science, software engineering, or self-study</td>
    </tr>

    <tr>
      <th>Experience</th>
      <td>Projects, internships, and real-world practice</td>
    </tr>

    <tr>
      <th>Training</th>
      <td>HTML, CSS, JavaScript, databases, and backend skills</td>
    </tr>
  </tbody>
</table>
```

Explanation

```
<table id="quickfacts">
```

Creates a table and gives it an `id`.

```
<caption>Software Developer Quick Facts</caption>
```

Names the table.

```
<th colspan="2">Quick Facts</th>
```

Creates one header cell that stretches across 2 columns.

```
<th>Median Pay</th>  
<td>$130,160 per year</td>
```

Uses `th` for the row label and `td` for the data.

40. Warm-Up Drills

Practice these small tasks before building a full page.

Drill 1: Write a Basic Table

Write a table with one header row and one data row.

Required elements:

- `table`
- `tr`
- `th`
- `td`

Drill 2: Add a Caption

Add a `caption` to your table.

Example:

```
<caption>Student Marks</caption>
```

Drill 3: Add `thead` and `tbody`

Separate your table into:

- heading section
- body section

Drill 4: Add `tfoot`

Add a summary row at the bottom.

Drill 5: Use `colspan`

Make one footer cell stretch across all columns.

Drill 6: Fix a Missing Closing Tag

Fix this:

```
<h2>Table Notes
<p>This paragraph should not be inside the heading.</p>
```

Correct version:

```
<h2>Table Notes</h2>
<p>This paragraph should not be inside the heading.</p>
```

41. Guided Coding Task

Create a file named:

```
sprint-9-course-comparison.html
```

Build a course comparison table.

Your page must include:

- full HTML boilerplate
- one `h1`
- one short paragraph
- one `table`
- one `caption`
- one `thead`
- one `tbody`
- one `tfoot`
- at least 3 course rows
- `th` for column headings

- `td` for data cells
- one `colspan`
- no table used for layout

Suggested Data

Course	Duration	Level	Main Skill
HTML	10 hours	Beginner	Structure
CSS	15 hours	Beginner	Styling
JavaScript	25 hours	Intermediate	Behavior

42. Challenge Task

After finishing the main table, add another small table that uses `rowspan`.

Example topic:

Frontend learning topics.

```
<table>
  <caption>Frontend Learning Topics</caption>
  <tbody>
    <tr>
      <th rowspan="2">Frontend Basics</th>
      <td>HTML</td>
    </tr>
    <tr>
      <td>CSS</td>
    </tr>
  </tbody>
</table>
```

Then explain in one sentence:

The `rowspan` attribute is used because Frontend Basics applies to both HTML and CSS.

43. Sprint 9 Revision Questions

Use these questions to test yourself.

Question 1

What is a table used for?

Answer:

A table is used to show data arranged in rows and columns.

Question 2

Should tables be used for page layout?

Answer:

No. Tables should be used for tabular data, not layout.

Question 3

What does `table` do?

Answer:

It wraps the whole table.

Question 4

What does `caption` do?

Answer:

It gives the table a title or explanation.

Question 5

Where should `caption` usually go?

Answer:

Directly after the opening `table` tag.

Question 6

What does `thead` contain?

Answer:

It contains the table header rows.

Question 7

What does `tbody` contain?

Answer:

It contains the main table data rows.

Question 8

What does `tfoot` contain?

Answer:

It contains footer, summary, or note rows for the table.

Question 9

What does `tr` mean?

Answer:

`tr` means table row.

Question 10

What does `th` mean?

Answer:

`th` means table header cell.

Question 11

What does `td` mean?

Answer:

`td` means table data cell.

Question 12

When should you use `th`?

Answer:

Use `th` when the cell is a heading or label for other cells.

Question 13

When should you use `td`?

Answer:

Use `td` for normal data values.

Question 14

What does `colspan` do?

Answer:

It makes a cell stretch across multiple columns.

Question 15

What does `rowspan` do?

Answer:

It makes a cell stretch down across multiple rows.

Question 16

Why should tables stay simple?

Answer:

Simple tables are easier to read, debug, maintain, and understand with assistive technology.

Question 17

What does it mean that browsers are forgiving?

Answer:

Browsers may still display broken HTML by guessing the intended structure.

Question 18

Why can forgiving browser parsing be dangerous?

Answer:

Because the page may look okay, but the actual HTML structure may be wrong.

Question 19

What does an HTML validator do?

Answer:

It checks HTML against standards and reports structural mistakes.

Question 20

What can DevTools Elements show you?

Answer:

It shows the HTML structure as the browser understands it.

Question 21

What can the Console show you?

Answer:

It can show errors, warnings, and broken path problems such as 404 errors.

Question 22

What is the difference between source HTML and parsed DOM?

Answer:

Source HTML is what you wrote. Parsed DOM is what the browser built from your HTML.

44. Sprint 9 Key Rules

Memorize these rules:

- Use tables for tabular data only.
 - Do not use tables for page layout.
 - Do not fake data tables with `div` elements.
 - Use `table` as the outer table container.
 - Use `caption` to name or describe the table.
 - Place `caption` near the start of the table.
 - Use `thead` for the heading area.
 - Use `tbody` for the main data area.
 - Use `tfoot` for footer or summary rows.
 - Use `tr` for each table row.
 - Use `th` for header cells.
 - Use `td` for normal data cells.
 - Put every `th` and `td` inside a `tr`.
 - Use `colspan` when a cell must stretch across columns.
 - Use `rowspan` when a cell must stretch across rows.
 - Keep tables simple.
 - Browsers may hide HTML mistakes by guessing.
 - A page that looks okay can still have invalid HTML.
 - Use a validator to check standards.
 - Use DevTools Elements to inspect browser-parsed structure.
 - Use DevTools Console to notice broken paths and errors.
 - Fix one HTML problem at a time.
-

45. Sprint 9 Mini Glossary

Table

A structure used to show data in rows and columns.

Example:

```
<table>...</table>
```

Tabular Data

Data that naturally fits into rows and columns.

Examples:

- course comparison
- price list
- timetable
- marks table

table

The outer container for a table.

```
<table>
...
</table>
```

caption

The title or explanation of a table.

```
<caption>Course Comparison</caption>
```

thead

The table header section.

```
<thead>...</thead>
```

tbody

The main table data section.

```
<tbody>...</tbody>
```

tfoot

The table footer or summary section.

```
<tfoot>...</tfoot>
```

tr

A table row.

```
<tr>...</tr>
```

th

A table header cell.

```
<th>Course</th>
```

td

A table data cell.

```
<td>HTML</td>
```

colspan

An attribute that makes a cell stretch across columns.

```
<td colspan="3">Summary note</td>
```

rowspan

An attribute that makes a cell stretch across rows.

```
<th rowspan="2">Frontend</th>
```

Validator

A tool that checks whether your HTML follows standards.

Example:

validator.w3.org

DevTools

Browser tools used to inspect and debug web pages.

Elements Tab

A DevTools tab that shows the page structure as the browser understands it.

Console Tab

A DevTools tab that shows messages, warnings, and errors.

DOM

The browser's internal structure of the HTML page.

Parsed DOM

The structure the browser creates after reading your HTML.

404 Error

An error meaning the browser could not find a requested file or page.

46. Final Sprint 9 Self-Check

You are ready to move on from Sprint 9 when you can do these without looking at notes:

- Explain what tabular data means.
- Explain why tables should not be used for layout.
- Write a basic table from memory.
- Add a meaningful `caption`.
- Use `thead`, `tbody`, and `tfoot` correctly.
- Use `tr`, `th`, and `td` correctly.
- Explain the difference between `th` and `td`.
- Use `colspan` correctly.
- Use `rowspan` correctly.
- Keep table structure simple.
- Fix a table where `td` elements are outside `tr`.

- Fix a table that uses `div` instead of `caption`.
- Fix a missing heading closing tag.
- Explain why browsers may hide HTML errors.
- Explain what an HTML validator does.
- Use DevTools Elements to inspect structure.
- Use Console to identify errors or broken paths.

47. Sprint 9 Completion Standard

Sprint 9 is complete when you can look at this broken code:

```
<table>
  <div>Course Table</div>
  <tr>
    <td>Name</td>
    <td>Hours</td>
  </tr>
  <p>HTML</p>
  <td>10</td>
</table>

<h2>Notes
<p>This is not meant to be heading text.</p>
```

And fix it to this:

```
<table>
  <caption>Course Table</caption>

  <thead>
    <tr>
      <th>Name</th>
      <th>Hours</th>
    </tr>
  </thead>

  <tbody>
    <tr>
      <td>HTML</td>
      <td>10</td>
    </tr>
  </tbody>
</table>
```

```
<h2>Notes</h2>
<p>This is not meant to be heading text.</p>
```

You should also be able to explain each fix:

- `caption` replaces the wrong `div`.
- `th` replaces `td` for headings.
- `HTML` and `10` are placed inside the same row.
- `td` is inside `tr`.
- `h2` is closed correctly.
- The paragraph is separate from the heading.

48. Final Mental Model

Remember this simple mental model:

```
table = the whole table
caption = table title
thead = heading area
tbody = main data area
tfoot = footer or summary area
tr = table row
th = heading cell
td = data cell
colspan = stretch across columns
rowspan = stretch across rows
validator = checks HTML rules
DevTools Elements = shows browser-parsed structure
Console = shows errors and broken paths
```

Main point of Sprint 9:

Write tables only for real data, keep table structure clean, and always check your HTML instead of trusting the browser to guess correctly.